

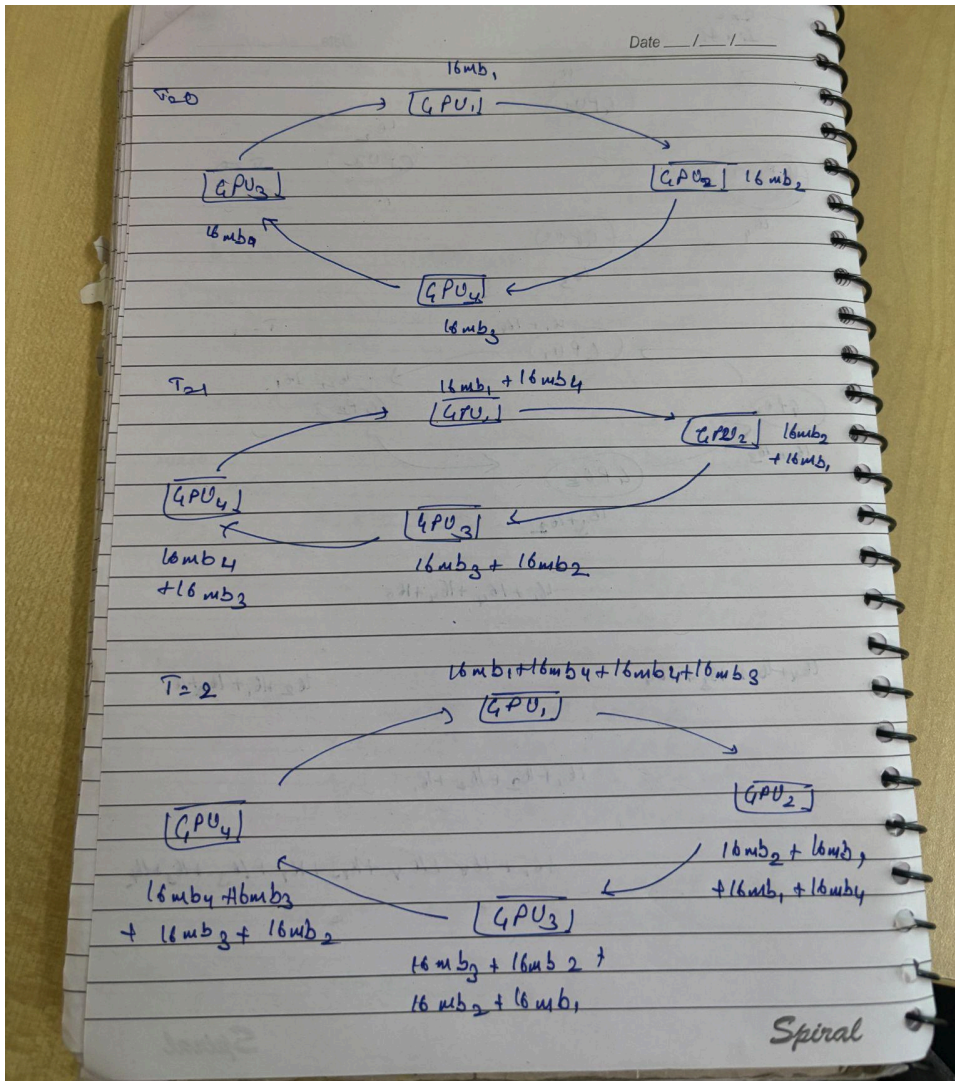
CSE587: Networks for AI/ML Systems

QUIZ 4 (30 mins) [Total: 16 points]

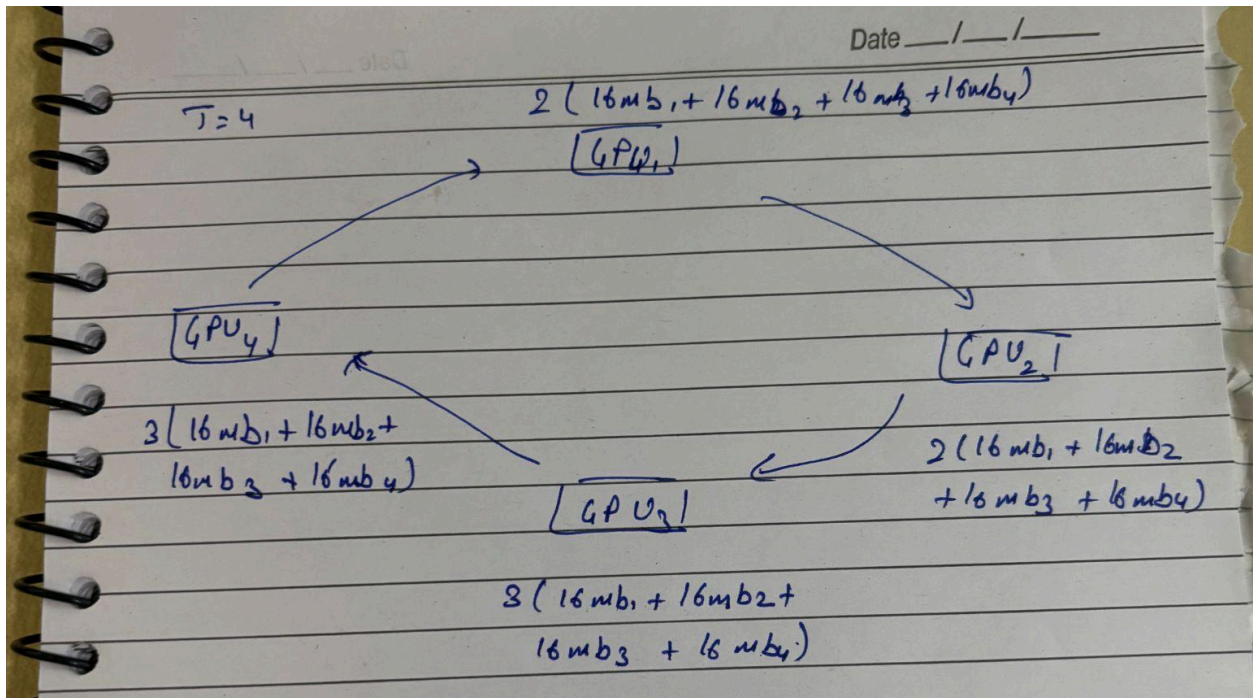
Oct 30, 2025

Q.1. Suppose there are 4 GPUs (ranks) arranged in a simple ring topology ($R_0 \rightarrow R_1 \rightarrow R_2 \rightarrow R_3 \rightarrow R_0$). Each rank needs to reduce 16MiB of data (1MiB (megabytes) = 2^{20} B). All links are equally fast; however, consider a faulty link between ranks $R_2 \rightarrow R_3$ due to which the speed is half of the others. You can split the message into "k" pieces ("chunks") to pipeline. Answer the following: **[3+2+2+0]**

a) With $k = 1$ (no splitting), describe the step by step process for reduce operation.



Ans:



Each rank has a buffer of size 'k=16 MiB'

R0 has buffer A0

R1 has buffer A1

R2 has buffer A2

R3 has buffer A3

[SOLUTION 1]

Step 0: Choose a root node, let's say we pick R0

Hop 1:

- R0→R1: A0 arrives at R1, R1 adds A0 ← A1 + A0
- R1→R2: A1 arrives at R2, R2 adds A1 ← A2 + A1
- R2→R3: A2 arrives at R3, R3 adds A2 ← A3 + A2
- R3→R0: A3 arrives at R0, R0 adds A3 ← A0 + A3

Hop 2:

- R1→R2: (A0+A1) arrives at R2, R2 adds A2 ← (A0+A1) + A2
- R2→R3: (A2+A1) arrives at R3, R3 adds A3 ← (A2 + A1) + A3
- R3→R0: (A3+A2) arrives at R0, R0 adds A0 ← (A3 + A2) + A0
- R0→R1: (A0+A3) arrives at R1, R1 adds A1 ← (A0 + A3) + A1

Hop 3:

- R3→R0 (Since we want to reduce at R0): (A1+A2+A3) arrives at R0, R0 adds A0 ← (A0+A1+A2+A3)

Note that each step's duration will be set by the slowest link.

CSE587: Networks for AI/ML Systems
QUIZ 4 (30 mins) [Total: 16 points]
Oct 30, 2025

[SOLUTION 2]

Step 0: Choose a root node, let's say we pick R0

Hop 1:

R0->R1: idle (root doesn't send for a Reduce)

R1->R2: A1 arrives at R2, R2 forwards (no add)

R2->R3: A2 arrives at R3, R3 forwards (no add)

R3->R0: A3 arrives at R0, R0 adds: $S \leftarrow A0 + A3$

Hop 2:

R0->R1: idle

R1->R2: idle (A1 already moved in Step 1)

R2->R3: A1 (forwarded from R2) arrives at R3, R3 forwards (no add)

R3->R0: A2 (forwarded from R3) arrives at R0, R0 adds: $S \leftarrow (A0 + A3) + A2$

Hop 3:

R0->R1: idle

R1->R2: idle

R2->R3: idle

R3->R0: A1 arrives at R0, R0 adds: $S \leftarrow (A0 + A3 + A2) + A1 = A0 + A1 + A2 + A3$

[Reduce is not AllReduce, so no need for All-Gather step]

b) Will some GPUs spend time waiting instead of continuously sending? Justify.

Ans: Waiting occurs because:

i) With one big 16 MiB piece, each hop takes one link-time to push that entire piece across.

ii) When the 16 MiB "token" passes through the slow hop (2->3), that hop takes twice as long as the others.

iii) Upstream GPUs (eventually 1-> 2) must wait because 2 can't forward onward faster than the slow hop drains. This creates pipeline bubbles: fast links go idle while the slow link is still busy. Net effect: the slow hop clocks the whole ring. Everyone else spends more time waiting than sending because a ring is only as fast as its slowest segment.

c) If we change to $k = 4$ equal pieces, will there be an improvement? Justify.

Ans: Chunking improves the utilization due to pipelining. Split 16 MiB into 4×4 MiB pieces. Now, as soon as the first 4 MiB piece leaves a GPU, that GPU can start sending the second piece behind it (pipelining). The startup time to fill the pipeline is shorter, and more links can be busy at the same time; less idle time while waiting for the slow hop to finish a huge piece. There is still the issue of the slow hop, but the overlap is improved because the ring spends more time in steady streaming and less time in long stalls. Chunking helps us turn one long wait into many short waits that overlap with useful work on other links.

CSE587: Networks for AI/ML Systems

QUIZ 4 (30 mins) [Total: 16 points]

Oct 30, 2025

- d) Even if we increase “k” by a lot, what ultimately limits the overall speed of the Ring AllReduce here, and why? (If you know Max-Flow Min-Cut Algorithm from your Algorithms course, this question is a piece of cake. Even if you do not know this, logic is all you need!) → FOR CRITICAL THINKING; NO MARKS ASSIGNED

Ans: Throughput is limited by the slowest hop (2->3). Making k bigger and bigger helps only up to a point: tiny chunks pay too much per-chunk overhead (latency, kernel launches, headers), so performance plateaus or drops. We can smooth idle time, but we cannot exceed the slow hop's rate. That's the min-cut of the ring.

Q.2. Why do we use techniques such as multi-pathing and packet-spraying? Explain the meaning of both terms. Give example scenarios (or use-cases) when you would employ multi-pathing, and when you would employ packet-spraying. Justify your answer. **[3+3]**

Ans.

Multi-pathing refers to the flow-level load balancing technique where different flows (or subflows) are routed across multiple paths between the same source and destination pair.

Each flow follows a consistent path (using hashing or connection IDs) — preserving packet order within the flow.

Packet-spraying refers to packet-level load balancing, where individual packets from a single flow are dynamically distributed across multiple paths. It helps balance the load more evenly in bursty traffic. However, it may cause packet reordering, requiring reordering logic at the receiver or NIC.

Example:

Multipathing: (1) Cloud service redundancy: A user has a long-lived connection to a cloud service. If one internet connection (e.g., the primary fiber line) fails, the Multi-pathing TCP (MPTCP) connection automatically fails over to the secondary connection (e.g., a backup cable line), maintaining the service.

(2) Data center storage access: In a Storage Area Network (SAN), a server needs highly available and reliable access to its storage. Multipath I/O (MPIO) can provide multiple physical paths from the server to the storage array for redundancy and failover.

Justification: Multi-pathing can intelligently manage and distribute data across paths with different latency and bandwidth characteristics. It uses a single congestion window across all subflows, allowing it to adapt to changing network conditions.

Packet-spraying: (1) Distributed AI/ML training: In a high-performance computing cluster for training large language models, thousands of nodes need to communicate with each other with extremely low latency. Packet-spraying is used to distribute traffic uniformly and minimize queueing delays.

(2) Real-time, latency-sensitive applications: High-frequency trading platforms or online gaming services require the lowest possible latency for every packet. Packet-spraying can distribute traffic to use all available bandwidth and bypass congested links immediately, reducing tail latency.

Justification: Packet-spraying's core function is to achieve a perfectly even distribution of load across all available network paths. By spreading packets instantly, it minimizes congestion and

CSE587: Networks for AI/ML Systems

QUIZ 4 (30 mins) [Total: 16 points]

Oct 30, 2025

reduces latency, which is critical for highly demanding workloads like AI/ML and parallel computing. Applications that use packet-spraying must be able to tolerate packet reordering, or use technologies like Forward Error Correction (FEC) or RDMA (Remote Direct Memory Access), which can handle out-of-order packets efficiently. This makes it less suitable for traditional TCP connections.

Q.3. Design a credit-based flow control mechanism for UET that ensures zero packet loss under 400 Gbps link utilization. Assume each flow control credit corresponds to 64B (i.e., 64 bytes) and the round-trip propagation delay is 800ns. Determine the minimum number of credits required to sustain the line rate. **[3]**

Ans.

$\text{Bandwidth} \times \text{RTT} = 400 \text{ Gbps} \times 800 \text{ ns} = 320 \text{ Kb} = 40 \text{ KB}$ (in flight bytes to fill the pipe)

$\text{Credit size} = 64 \text{ B} \rightarrow 40 \text{ KB} / 64 \text{ B} = 640 \text{ credits required}$
